

Apunte de Clase: Microarquitectura - Pipeline Escalar en RISC

Introducción

Una de las técnicas fundamentales para aumentar el rendimiento de un procesador sin aumentar la frecuencia de reloj es el pipeline (tubería o línea de montaje). Esta técnica introduce paralelismo a nivel de instrucción permitiendo que múltiples instrucciones se ejecuten de manera solapada en el tiempo.

La analogía clásica es una línea de montaje industrial: en lugar de que un solo operario haga todo el trabajo sobre un producto, se divide el proceso en etapas y cada operario se especializa en una. El producto avanza de una etapa a la siguiente, permitiendo que múltiples productos estén en proceso simultáneamente.

En este apunte nos centraremos en pipeline escalar (una instrucción por etapa en cada ciclo) sobre arquitecturas RISC (Reduced Instruction Set Computer), que por su simplicidad son ideales para implementar pipelines eficientes.

1. Concepto de Pipeline

1.1 Definición básica

El pipeline consiste en segmentar un circuito (el procesador) en múltiples unidades funcionales (UF), cada una con una tarea específica. En lugar de tener un único circuito enorme que ejecuta una instrucción completa de una vez, se separa la ejecución en etapas.

Consecuencias:

- Se pierde latencia por instrucción individual (una instrucción tarda más en completarse porque debe atravesar todas las etapas).
- Se gana en throughput (rendimiento): después de que las primeras n instrucciones llenan el pipeline, se completa una instrucción por ciclo de reloj.

1.2 Analogía con una línea de montaje

Imaginemos una fábrica de autos:

- Sin pipeline: un solo equipo construye un auto completo (chasis, motor, pintura, etc.) y luego comienza el siguiente. Solo se produce un auto por vez.
- Con pipeline: el proceso se divide en etapas (montaje de chasis, instalación del motor, pintura, control de calidad). Cada etapa tiene su propio equipo. Mientras un auto está en pintura, otro ya está recibiendo el motor. Así se producen múltiples autos simultáneamente.

En un procesador ocurre exactamente lo mismo con las instrucciones.

2. Ciclo de instrucción típico de 5 etapas

La mayoría de los pipelines RISC clásicos (como el MIPS) utilizan 5 etapas estándar:

Etapa	Nombre	Función
IF	Instruction Fetch	Leer la siguiente instrucción desde la memoria de instrucciones (cache).
ID	Instruction Decode / Register Fetch	Decodificar la instrucción y leer los operandos de los registros.
EX	Execute / Effective Address	Realizar la operación de la ALU (suma, resta, AND, etc.) o calcular dirección efectiva para loads/stores.
MA	Memory Access	Acceder a la memoria de datos (solo para instrucciones load o store).
WB	Write Back	Escribir el resultado en el registro destino (si corresponde).

2.1 Comportamiento en el tiempo

En un procesador sin pipeline, una instrucción ocupa el procesador completo durante todo su ciclo (por ejemplo, 5 unidades de tiempo). En un pipeline, cada etapa toma 1 ciclo de reloj y las instrucciones se solapan:

Ciclo	IF	ID	EX	MA	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I4	I3	I2	I1	
5	I5	I4	I3	I2	I1
6	I6	I5	I4	I3	I2

- A partir del ciclo 5, se completa una instrucción por ciclo.
- El tiempo por instrucción promedio = (tiempo sin pipeline) / (cantidad de etapas), idealmente.

3. Instrucciones RISC y sus etapas

En una arquitectura RISC típica, cada tipo de instrucción usa un subconjunto de las etapas:

Tipo de instrucción	Etapas que utiliza	Ejemplo (MIPS)
Aritmético-lógica (ALU)	IF → ID → EX → WB	DADD R1, R2, R3
Load (carga desde memoria)	IF → ID → EX (cálculo de dirección) → MA → WB	LD R1, offset(R2)
Store (almacenamiento)	IF → ID → EX (cálculo de dirección) → MA (escribe en memoria)	SD R1, offset(R2)
Branch (salto condicional)	IF → ID → EX (compara y calcula destino)	BEQ R1, R2, offset

Observación: En arquitecturas RISC no hay instrucción MOV separada; se usa DADD con un cero o un OR con cero.

4. Riesgos del Pipeline (Pipeline Hazards)

Un riesgo (hazard) es cualquier situación que impide que la siguiente instrucción se ejecute en el ciclo de reloj programado, rompiendo la fluidez del pipeline. Se clasifican en tres tipos:

4.1 Riesgos Estructurales (Structural Hazards)

- Causa: El hardware no tiene suficientes recursos duplicados para soportar simultáneamente todas las combinaciones de instrucciones en el pipeline.
- Ejemplo: Una única memoria unificada (instrucciones y datos) no permite que una instrucción haga un load (acceso a datos) mientras otra hace fetch de la siguiente instrucción.
- Consecuencia: Se produce un stall (detención) o burbuja (bubble) en el pipeline. La instrucción se detiene hasta que el recurso esté disponible, y las instrucciones posteriores también se detienen.
- Solución típica: Duplicar recursos (ej: memorias separadas para instrucciones y datos – Harvard architecture). En pipelines superescalares se duplican ALUs, etc.

4.2 Riesgos de Datos (Data Hazards)

- Causa: Una instrucción depende del resultado de otra instrucción previa que aún no ha terminado de escribirse.
- Ejemplo clásico del documento:

assembly

DADD R1, R2, R3 ; escribe en R1

DSUB R4, R1, R5 ; necesita leer R1 (aún no está disponible)

AND R6, R1, R7 ; también depende de R1

OR R8, R1, R9 ; también

XOR R10, R1, R11 ; también

- Problema: La instrucción DSUB intenta leer R1 en la etapa ID, pero DADD aún no ha escrito R1 (lo hará en WB varios ciclos después).

Solución 1: Forwarding (Caminos de Adelanto)

- Idea: El resultado de la ALU (etapa EX) está disponible mucho antes de que se escriba en el banco de registros (WB). Se puede adelantar ese valor directamente a la entrada de la ALU de la siguiente instrucción.
- Mecanismo: La lógica de control detecta que el destino de la operación anterior es fuente de la operación actual. Entonces, en lugar de leer el registro (que aún está viejo), toma el valor que está saliendo de la ALU en ese momento.
- Ventaja: Permite resolver la mayoría de las dependencias sin stalls.

Solución 2: Stalling (solo si forwarding no es posible)

- Algunas dependencias no pueden resolverse con forwarding (ej: una instrucción que necesita un valor que recién estará disponible después de un load desde memoria). En esos casos, se introduce un stall.

4.3 Riesgos de Control (Control Hazards)

- Causa: Las instrucciones de branch (saltos condicionales) pueden cambiar el flujo del programa. El pipeline no sabe si debe seguir con la siguiente instrucción secuencial o saltar a otra dirección.
- Consecuencia: Si se predice mal, las instrucciones ya ingresadas al pipeline son incorrectas y deben descartarse (flushed), perdiendo ciclos.

Soluciones posibles:

1. Stall siempre (más simple pero ineficiente): El pipeline se detiene hasta que se resuelve la condición del branch. Penalidad fija (ej: 2 ciclos).
2. Predicción "siempre no tomado" (Predicted-not-taken): Se asume que el branch no se tomará y se sigue con la instrucción siguiente. Si el branch sí se toma, hay que descartar la instrucción equivocada y buscar la correcta.
3. Predicción "siempre tomado" (Predicted-taken): Similar al anterior pero asumiendo que sí se tomará.
4. Predicción dinámica de saltos: El procesador aprende el comportamiento de cada branch (usando un historial) y predice en consecuencia. Puede alcanzar más del 95% de aciertos.

5. Optimización por compilador: El compilador reorganiza el código para minimizar el impacto de los branches (ej: moviendo código fuera del bucle, o usando predicción estática basada en hints).

5. Técnicas de Optimización Relacionadas

5.1 Desenrollado de bucles (Loop Unrolling)

- Objetivo: Reducir la cantidad de branches (comparaciones y saltos) en un bucle, aumentando el tamaño del código pero mejorando el rendimiento del pipeline.
- Método: Replicar el cuerpo del bucle varias veces y ajustar los índices.
- Ejemplo conceptual (del documento):

Código C

```
// Bucle original (1000 iteraciones)
for(i = 0; i <= 1000; i++) {
    a[i] = a[i] + 5;
}

// Desenrollado (50 iteraciones, cada una procesa 50 elementos)
for(i = 0; i <= 1000; i += 50) {
    a[i]    = a[i]    + 5;
    a[i+1] = a[i+1] + 5;
    // ... repetir 50 veces
    a[i+49] = a[i+49] + 5;
}
```

- Requisitos: Las iteraciones deben ser independientes (no deben haber dependencias de datos entre ellas). Se usan registros distintos para evitar agregar dependencias artificiales.
- Ventaja: Se eliminan muchas comparaciones y branches, reduciendo los riesgos de control.
- Desventaja: Aumenta el tamaño del código (mayor uso de cache de instrucciones).

5.2 Planificación estática (por el compilador)

El compilador reorganiza las instrucciones para minimizar los stalls, aprovechando el conocimiento del pipeline de la CPU objetivo. Es una técnica complementaria al forwarding.

6. Ventajas y Desventajas del Pipeline

Ventajas

- Mayor throughput (rendimiento): En promedio, se completa una instrucción por ciclo de reloj después del llenado inicial.
- Mejor utilización del hardware: Las unidades funcionales trabajan simultáneamente.
- Fundamento de procesadores modernos: Todos los procesadores de alto rendimiento usan pipelines (y versiones superescalares, con múltiples pipelines).

Desventajas

- Aumento de la latencia por instrucción individual: Una instrucción aislada tarda más en completarse (tiene que pasar por todas las etapas).
- Overhead de implementación: Los registros de segmentación (flip-flops entre etapas) añaden costo y consumo.
- Riesgos (hazards): Los stalls reducen la ganancia ideal. En la práctica, la aceleración real es menor a la cantidad de etapas.
- Complejidad de diseño: La lógica de control de forwarding, detección de hazards y predicción de saltos es considerable.

7. Tabla Resumen de Riesgos y Soluciones

Tipo de riesgo		Causa		Solución típica	
Estructural		Recurso insuficiente	compartido	Duplicar (memorias múltiples ALUs)	recursos separadas,
Datos (RAW, WAR, WAW)		Dependencia instrucciones	entre	Forwarding + stalls mínimos	
Control (branch)		Cambio impredecible	de flujo	Predicción de saltos (estática o dinámica) + desenrollado de bucles	

Nota: Los riesgos de datos se subclasifican en RAW (Read After Write) – el más común, WAR (Write After Read) y WAW (Write After Write) – estos últimos menos frecuentes en pipelines simples.

El pipeline escalar es una técnica fundamental en la microarquitectura de procesadores RISC. Permite aumentar el rendimiento explotando el paralelismo a nivel de instrucción sin necesidad de múltiples núcleos. Sin embargo, introduce riesgos (estructurales, de datos y de control) que deben ser gestionados mediante técnicas como forwarding y predicción de saltos. Los compiladores también juegan un rol importante (desenrollado de bucles,

planificación estática). Comprender el pipeline es esencial para cualquier estudiante de sistemas que desee optimizar software de bajo nivel o entender el funcionamiento interno de los CPUs modernos.